

README / Setup Guide

Overview

This application is an employee leave management system designed to streamline the process of submitting, approving, and tracking leave requests within an organization. It provides a web-based interface for employees to submit leave requests and view their leave status. The system ensures that leave requests are processed efficiently and securely, adhering to organizational policies.

Key Features

- **Leave Request Submission:** Employees can submit leave requests through a user-friendly web interface, specifying the leave type, start date, and end date.
- **Leave Status Tracking:** Employees can view the status of their leave requests, including pending, approved, and rejected statuses.
- **Leave Balance Monitoring:** Employees can check their remaining leave balance for different leave types.

Prerequisites

- **Java 11+**
- **PostgreSQL 13+**
- **Node.js 16+**
- **npm 8+**
- **Docker (optional for local development)**
- **Linux/Mac/Windows**

Project Structure

The repository is structured as follows:

- **backend/:** Contains the server-side code, including RESTful APIs, business logic, and database interactions. Key frameworks include Spring Boot for the backend framework and JPA for database access.
- **frontend/:** Contains the client-side code, including the web interface for submitting and viewing leave requests. Key frameworks include React for the frontend framework and Axios for API communication.
- **shared/:** Contains shared utilities and configurations used by both the frontend and backend.
- **docs/:** Contains documentation, including API documentation and user guides.

Installation & Setup

Backend Setup

1. Clone the repository: `git clone <url>`
2. Navigate to the project: `cd employee-leave-management`
3. Create virtual environment: `python -m venv venv`
4. Activate: `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)
5. Install dependencies: `pip install -r backend/requirements.txt`
6. Set up PostgreSQL database: Ensure PostgreSQL is running and create a database named `leave_management`.
7. Configure environment variables: Copy `.env.example` to `.env` and update with your database credentials and JWT secret.

Frontend Setup

1. Navigate to frontend: `cd frontend`
2. Install dependencies: `npm install`
3. Build/prepare assets: `npm run build`

Configuration

Required Environment Variables

- **DB_HOST**: The hostname of the PostgreSQL database (e.g., `localhost`).
- **DB_PORT**: The port number of the PostgreSQL database (e.g., `5432`).
- **DB_NAME**: The name of the PostgreSQL database (e.g., `leave_management`).
- **DB_USER**: The username for the PostgreSQL database (e.g., `leave_user`).
- **DB_PASSWORD**: The password for the PostgreSQL database.
- **JWT_SECRET**: A secret key for signing JWT tokens (e.g., `my_secret_key`).

Optional Environment Variables

- **PORT** (default: `5000`): The port number on which the backend server will run.

Running the Application

Starting the Backend

1. Ensure virtual environment is activated.
2. Set environment variables: `export DB_HOST=localhost, export DB_PORT=5432, etc.`

3. Start server: `python backend/app.py`
4. Verify: Backend is running on `http://localhost:5000`.

Starting the Frontend

1. Navigate to frontend directory: `cd frontend`
2. Start dev server: `npm run dev`
3. Open browser: `http://localhost:3000` (or displayed URL)
4. Verify: UI loads and connects to backend.

Verification Checklist

- ☐ Backend health check responds at `GET /api/health` with 200 OK.
- ☐ Frontend loads without console errors.
- ☐ Basic workflow completes end-to-end (e.g., submitting and approving a leave request).
- ☐ API endpoints respond with expected data format.

Troubleshooting

Backend Issues

- **Port already in use:** Change PORT env var or kill process on port 5000.
- **Module not found:** Run `pip install -r backend/requirements.txt` again.
- **Database connection error:** Verify connection string in `.env`, check service is running.
- **Authentication error:** Verify API keys/tokens in `.env` are correct and not expired.

Frontend Issues

- **Blank page or won't load:** Check browser console for errors, ensure backend is running.
- **API connection error:** Verify backend URL in config, check CORS settings.
- **Module resolution error:** Delete `node_modules` and run `npm install` again.
- **Build fails:** Ensure Node.js version matches requirements, clear cache: `npm cache clean --force`.

Common Workflows

Submit a Leave Request

1. Navigate to the leave request submission page.
2. Fill in the leave request form with employee ID, leave type, start date, and end date.

3. Click submit.
4. Verify: A new leave request is created with status "PENDING".

Approve a Leave Request

1. Navigate to the leave request details page.
2. Click the approve button.
3. Verify: The leave request status is updated to "APPROVED".

Deployment Notes

- Ensure environment variables are set correctly in the production environment.
- Use a secure method for managing secrets, such as environment variables or a secrets manager.
- Consider using a load balancer and scaling the application based on traffic.

Getting Help

- Check logs: Backend logs in console, frontend logs in browser DevTools.
- Review error message and stack trace for clues.
- Check environment variables are set correctly.
- Refer to the API Documentation and User Guide for more details.